

Cashflow2.0 — Web Migration: Performance, Hosting Tiers, and Platform Options

A summary of the improvements made during the web migration, current hosting trade-offs, and options for the future.

Summary

This document covers four things: (1) how much faster the app got during the recent restructuring work and why, (2) the trade-offs of the current free hosting tiers versus paid upgrades, (3) what's involved in giving the site a custom domain, and (4) what a native mobile app would actually add on top of the website, which already mimics the native app experience on phones.

Short version: the app is meaningfully faster than before, especially for repeat/bulk categorisation work. The current free hosting is fine for personal/family use but has two specific soft spots worth knowing about — a short delay after inactivity (Render) and a harder weekly pause plus no automatic backups (Supabase) — each fixable independently for about £20/month if it ever becomes worth it. A custom domain is a nice-to-have, not a requirement. And a native app's real remaining advantages — home screen presence and push notifications — can both be substantially closed without building a native app at all, via a lightweight upgrade to the existing website.

1. Performance improvements: before vs. after

The following are approximate, illustrative figures based on how each technique scales — not measured production benchmarks — but they reflect the real, structural reason each change helps.

1.1 Merchant matching (Aho-Corasick vs. checking each merchant individually)

Previously, matching a transaction description against known merchants meant checking it against every merchant name one by one — the more merchants the system has learned, the slower every single check becomes.

- At a small merchant count (~50 known merchants): roughly 1–2ms per description either way — the difference isn't noticeable yet.
- At a large, mature merchant count (~5,000+): the old approach could balloon to 100–200ms per description, since it's checking hundreds of times more candidates. The new approach stays at roughly 1–2ms regardless, because it scans the description once rather than looping over the merchant list.
- Net effect: roughly 10–30x faster at modest scale, and potentially 1,000x+ faster once the merchant dictionary has grown large — the gap widens as the system learns more, rather than the old approach staying flat.

1.2 Batched database writes (one round trip vs. many)

Writing newly-learned categorisations to the database used to risk one round trip per record. Each round trip carries real overhead — network latency plus the cost of committing a change.

- Illustrative example: 1,000 newly-learned records at roughly 5–15ms of overhead each could cost 5–15 seconds in pure database chatter alone, before counting any actual work.

- The batched approach sends the same 1,000 records in a single write, cutting that overhead to well under half a second — the size of the batch stops being the bottleneck.

1.3 Within-run learning during large categorisation jobs

When categorising a big batch of new, never-seen-before transactions, the system now re-checks its own progress between smaller chunks — so if it learns a merchant or pattern in chunk 1, chunk 2 can benefit from that immediately, without needing to ask the AI again.

- Illustrative example: the first chunk of a large upload (all genuinely new) might take ~20–30 seconds, dominated by AI response time.
- By the fourth or fifth chunk of the same upload, a meaningful share of transactions (commonly somewhere in the 30–70% range, highly dependent on the data) get resolved instantly by the cheaper matching tiers, cutting that chunk's time to roughly 8–12 seconds.
- This means categorisation speed increases as a single upload progresses — it gets faster, not slower, the more it has to do.

1.4 Codebase restructuring — a different kind of speed

Splitting the backend and frontend's largest files into smaller, clearly-named modules doesn't change how fast the app runs for users at all — it changes how fast it is to work on.

- Illustrative example: locating and safely changing one piece of logic inside a single ~1,000-line file might take 10–15 minutes of re-reading surrounding context to be sure nothing else breaks.
- In the restructured version, the same task often takes 2–3 minutes, since the file's own name and scope already narrow down where to look.

This matters for anyone maintaining or extending the app going forward, including future development work — it's a real time saving, just not one a user of the app would ever notice directly.

2. Hosting tiers: current setup and trade-offs

The app currently runs on two separate free-tier services: Render (backend/API) and Supabase (database). These are independent decisions — either can be upgraded on its own without needing to upgrade the other.

2.1 Render (backend hosting)

Tier	Cost	What changes
Free	£0	Backend goes to sleep after 15 minutes of no activity; next request takes ~30–60 seconds to wake it back up; 512MB RAM / 0.1 CPU; 750 hours/month compute included; 100GB/month bandwidth; no card required.
Starter	~£6/mo per service	Eliminates the sleep/wake delay entirely — the backend stays on and responds instantly at all times.
Standard	~£20/mo	More RAM/CPU and horizontal autoscaling — built for genuine production traffic; well beyond what this app currently needs.

What extra RAM/CPU actually helps with here: the backend's heaviest moments are large categorisation batches — matching, scoring, and holding a batch of transactions in memory while waiting on the AI. More RAM matters most for very large uploads (multiple years of statements at once) or large in-memory caches; more CPU speeds up the matching/scoring work itself. At current usage levels (personal or family use), the free tier's limits are very unlikely to be a real constraint — this only becomes relevant with many concurrent users or unusually large uploads.

2.2 Supabase (database)

Tier	Cost	What changes
Free	£0	500MB database, 1GB file storage, 5GB egress/month, up to 2 active projects. The project fully pauses after 7 days with no API activity — not a delay, an outage until manually resumed. No automated backups included.
Pro	~£20/mo	8GB database, 100GB storage, no auto-pause, daily backups included, higher usage limits across the board.
Team	~£475/mo	Compliance certifications (SOC2/ISO), longer backup retention — not relevant at this project's scale.

The Supabase free-tier risk is meaningfully different from Render's, and worth understanding as two separate things:

- Render's free tier costs seconds — the backend wakes up automatically the moment someone visits after a period of inactivity.
- Supabase's free tier costs an outage — if nobody uses the app for 7 days, the database stays offline until someone manually logs into the Supabase dashboard and resumes it. There's no automatic wake-up on the next visit.
- Supabase's free tier also has no automated backups at all. If something went wrong with the data — accidental deletion, a bug, corruption — there is currently no safety net to restore from. This is a real, separate risk from the pause behaviour, and arguably the more important of the two to plan around for a finance app specifically.

Because these two services are billed and upgraded independently, it's entirely possible to upgrade only the one that matters most. Given the risks above, Supabase's Pro tier (fixing both the pause risk and the backup gap at once) is arguably the higher-priority upgrade of the two if the app is used only occasionally, while Render's free tier's short wake-up delay is a much smaller practical concern by comparison.

2.3 A note on backups specifically

Given Supabase's free tier includes no automated backups, a lower-cost interim option worth considering before committing to the £20/month Pro upgrade: a scheduled export of the database (e.g. via a GitHub Actions workflow running on a timer, similar to the deployment workflow already in place) that periodically saves a copy of the data somewhere safe. This wouldn't remove the weekly pause risk on its own, but it would close the “no safety net” gap at little to no ongoing cost, as a stepping stone before or instead of a full Pro upgrade.

This works by using GitHub Actions (already in use for deployment) to run `pg_dump` — Postgres's own export tool — against the database on a schedule, saving the resulting file either as a GitHub Actions artifact or to external storage. Restoring from it later means running the saved file back into a fresh, empty database using `psql`, verifying the data looks correct, and only then pointing the app at it or copying the specific missing data back — this is a manual process, not a one-click restore, and is worth testing at least once so it's a known, practised process rather than untested when it's actually needed.

2.4 A workaround for the weekly pause: a scheduled “keep-alive” ping

Separately from backups, the 7-day auto-pause itself can be substantially mitigated with a similar approach: a small GitHub Actions workflow that pings the database every few days, which resets Supabase's inactivity timer and prevents the pause entirely. Any database request counts as activity — a simple read, not necessarily a written row — so a lightweight scheduled check is enough.

Important caveat: this is a widely used, generally reliable workaround, but it is not an officially guaranteed feature of Supabase's free tier — it relies on the platform continuing to treat a scheduled ping the same as genuine usage, which has occasionally been reported to behave inconsistently. It should be treated as a strong mitigation to combine with the backup workflow above, not as an absolute guarantee that pausing can never happen. The two together cover both angles: the ping makes pausing unlikely, and the backup ensures that even if a pause (or, in the worst case, an extended pause leading to eventual deletion) does occur, a recent copy of the data still exists to restore from.

2.5 Setting up and running both workflows

Both the backup workflow and the keep-alive workflow are implemented as GitHub Actions files placed in `.github/workflows/` (`supabase-backup.yml` and `supabase-keep-alive.yml`). Neither runs automatically until its required secrets are added:

- `SUPABASE_DB_URL` — the full Postgres connection string (Project Settings → Database → Connection string, “URI” format, with the real password filled in) — required for the backup workflow.
- `SUPABASE_URL` — the project's API URL (Project Settings → API) — required for the keep-alive workflow.
- `SUPABASE_ANON_KEY` — the public “anon” key (same API settings page) — required for the keep-alive workflow.

These are added under the repository's Settings → Secrets and variables → Actions → New repository secret. Once added, either workflow can be tested immediately without waiting for its schedule: open the Actions tab, select the workflow, and use the “Run workflow” button to trigger it manually. A successful run shows green; a failed one shows red with the specific error in the logs, which is deliberately treated as a visible failure rather than a silent pass — a broken key or an already-paused project should be noticeable, not hidden.

2.6 An important, not-yet-resolved security item: Row Level Security (RLS) and the anon key

While setting up the keep-alive workflow, a real, separate security question surfaced that is worth resolving on its own merits, independent of the workflow itself: whether Row Level Security (RLS) is enabled on the database's tables. As of this document, it is not yet enabled — this is flagged here as an open item, not something already fixed.

What RLS is and why it matters: Supabase's anon (public) API key is, by design, exposed in the deployed frontend's JavaScript — anyone can view the page source and find it. This is expected and fine on its own, PROVIDED Row Level Security is enabled on every table. RLS is what actually restricts what that key can do; without it enabled, a table has no row-level access restriction at all for requests made via Supabase's own REST API using that key, meaning anyone who extracted the key from the frontend could potentially query — or in some cases write to — the underlying tables directly, bypassing the application's own backend and authentication entirely.

Why this is very likely safe to enable without breaking anything: the application's Flask backend does not talk to the database through Supabase's REST API at all — it connects directly to Postgres using its own connection string, almost certainly authenticating as a privileged role (commonly the default postgres user Supabase provisions). Postgres's own rule is that privileged/superuser roles bypass Row Level Security entirely, regardless of what policies exist. This means enabling RLS with zero policies defined per table should: fully block the anon key from reading or writing any table via Supabase's API (the intended fix), while leaving the backend's own direct queries completely unaffected, since the backend's connection role isn't subject to RLS in the first place.

The one thing worth confirming before flipping this on, rather than assuming: which exact database role the backend's own connection string authenticates as. If it uses the standard postgres superuser role (the default for a fresh Supabase project), enabling RLS is safe by the reasoning above. If a separate, more restricted role was created for the backend at some point, that specific role's `BYPASSRLS` privilege should be checked (and granted if missing) before enabling RLS, to avoid inadvertently blocking the backend's own access.

Recommended next step: enable RLS on every table via the Supabase dashboard (Table Editor clearly flags any table where it is currently off), leave policies empty initially (the safe default — no access via the anon key at all), confirm the backend still works normally afterward, and only add specific policies later if a genuine need arises for the anon key to access certain data directly.

3. Custom domain options

The site currently runs at a free github.io address. This is a fully valid, permanent option on its own — moving to a custom domain is purely cosmetic (a nicer, more memorable web address) and is not required for anything the app does functionally.

3.1 Option A — buying a new domain

Cost is typically £5–15/year depending on the domain ending chosen (.com tends to be pricier; .dev, .app, .xyz are often cheaper). This path is the simplest: the domain is registered somewhere with full, direct control over its settings, and pointing it at the website is a short, well-documented process with no coordination needed with anyone else.

- Choose a registrar (e.g. Namecheap, Porkbun, Google Domains, GoDaddy) and search for an available domain name.
- Purchase the domain.
- In the registrar's DNS management panel, add the required records: for an apex domain (e.g. cashflow2.com), four A records pointing to GitHub's IP addresses (185.199.108.153, 185.199.109.153, 185.199.110.153, 185.199.111.153); for a subdomain (e.g. app.cashflow2.com), one CNAME record pointing to ideas-of-stuff-to-learn.github.io.
- In the GitHub repository's Settings → Pages → Custom domain, enter the chosen domain and save.
- Wait for DNS propagation — usually minutes, occasionally up to 24 hours.
- Once GitHub verifies the domain, it automatically issues a free HTTPS certificate (may take a few extra minutes after DNS resolves).
- Enable “Enforce HTTPS” in the same Pages settings once the certificate is ready.
- Update the backend's CORS allowed-origins list to include the new domain, and redeploy the backend.
- Update the frontend's build configuration (base path and router basename) — with a custom domain, the site serves from the root rather than a subfolder, so these can be simplified back to a plain “/”.

3.2 Option B — using a domain already purchased through WordPress

This path is more involved, since WordPress.com manages domains through its own dashboard rather than a plain domain registrar interface. Making the domain point somewhere outside WordPress generally requires either changing where its settings are managed from entirely, or editing specific records directly if WordPress's plan allows it.

- Confirm which WordPress plan the domain was purchased under — this determines whether DNS records can be edited directly, or whether nameservers need changing instead.
- Log into the WordPress.com account and locate the domain under Upgrades → Domains.
- Check whether the domain has an active email service tied to it (WordPress-hosted, or an external provider routed through it) — note the existing MX records before changing anything, so email can be restored afterward.
- If direct DNS record editing is available on the plan: add the same records as Option A (4 A records or 1 CNAME) directly in WordPress's DNS panel, skipping the nameserver change entirely.
- If direct editing isn't available: change the domain's nameservers away from WordPress's own, pointing instead to a third-party DNS provider (e.g. Cloudflare's free plan). This requires re-adding ALL existing records the domain relies on at the new provider first — not just the new ones — including the preserved MX records, or email will break.
- Wait 24–48 hours for the nameserver change to fully propagate globally.
- Once propagated, add the GitHub Pages records (as in Option A) at the new DNS provider.
- Verify the domain in GitHub's Settings → Pages → Custom domain, same as Option A.

- Test email delivery specifically (send a test email to and from the domain) before considering the migration complete — this is the step most likely to silently break.
- Complete the same backend CORS and frontend config updates as Option A.

Real risks worth knowing about upfront: changes can take up to 24–48 hours to fully take effect, during which the domain may behave inconsistently; if the domain is also used for email through WordPress, changing its settings risks breaking that email delivery unless handled carefully as above; and access needs to be either shared or coordinated with whoever manages the WordPress account, adding back-and-forth that Option A doesn't require.

3.3 Recommendation

Staying on the free github.io address is a perfectly reasonable long-term choice if a nicer web address isn't a priority. If it is, buying a new domain (Option A) is simpler, faster, and fully within one party's control — worth it only if a small recurring cost is acceptable. Using the client's existing WordPress domain (Option B) avoids that cost but trades it for real coordination overhead and a small risk of disrupting existing email, and is best done only with clear WordPress dashboard access and a plan for preserving email settings.

4. Native mobile app vs. the website

The website has already been built to look and feel like a native app when opened on a phone — so the usual arguments for building a native app on top of that need to be weighed honestly against what's actually still missing.

4.1 Arguments that don't meaningfully apply here

- “Native apps feel more like an app” — already solved; the website automatically switches to an app-style layout on phone-sized screens.
- “Native apps are faster” — for a data-driven app like this (not graphics- or game-heavy), the real-world performance difference is negligible.

4.2 Genuine, structural advantages a native app still has

- **Home screen presence — a real app sits on the home screen and opens in one tap; a website requires opening a browser first, which is a real, meaningful difference in how often something actually gets used day to day.**
- **Offline capability** — a native app can be built to work, at least partially, without an internet connection, using a real on-device database. The website currently has no offline story at all.
- **Push notifications** — a native app can notify someone even when it isn't open (e.g. “your new transactions have been categorised”). This is possible on the web too, but notably weaker on iPhone specifically, since Apple's web-push support is newer and more limited than Android's.
- **App store presence** — being listed in the App Store or Play Store adds legitimacy and discoverability, relevant mainly if the app were ever meant for people outside the immediate family.
- **Deeper device integration** — biometric login (Face ID / fingerprint) feels more natural and is more broadly supported in a native app than the web equivalent.

4.3 A middle path: these features can be added to either side

Several of the genuine native-app advantages can be added to either the existing website or a new native app, without necessarily needing both. The right choice depends on which starting point makes more sense — the sections below lay out what adding each feature actually looks like on each side.

4.4 Adding these features to a native app (if one were built)

- Home screen presence — automatic. This is simply the default behaviour of any native app; no extra work is needed.
- Push notifications — fully supported on both iOS and Android natively, including rich notifications and background delivery. This is the one area where a native app is strictly the fuller solution, with no caveats.
- Offline capability — native apps can use a real on-device database (e.g. SQLite) for genuine, full offline functionality, including viewing history and queuing actions to sync once reconnected — a stronger offline story than a web-based one can achieve.
- Biometric login — direct access to the OS-level Face ID / Touch ID / fingerprint APIs gives the smoothest possible integration.

4.5 Adding these features to the existing website (PWA + WebAuthn upgrade path)

- Home screen presence → turn the website into a Progressive Web App (PWA). A small amount of configuration lets someone “Add to Home Screen” from their browser, after which it opens full-screen with its own icon — visually indistinguishable from a native app at a glance. Closes this gap almost completely.
- Push notifications → PWAs can support web push notifications too. This works nearly as well as native on Android; on iPhone it's more limited, since Apple's web-push support is newer and more restricted than Android's — the one place the web genuinely can't fully match native.
- Offline capability → a service worker can cache the most recently loaded data, so the app shows something useful even with no connection, though it couldn't process brand-new uploads offline. Partial, read-mostly offline support, not full offline editing/sync.
- Biometric login → a web technology called WebAuthn (the same approach banks use on their websites) can add Face ID/fingerprint login to the existing website, using the same underlying OS biometric prompt, just triggered from the browser instead of a native API. Closes this gap almost completely.

Taken together, a PWA-plus-WebAuthn upgrade to the existing website would close most of the remaining native-app gap for a small fraction of the cost and time of a full React Native rebuild and app-store submission process — worth treating as the natural next step if any of these specific gaps (rather than “wanting an app” in general) turn out to matter.

4.6 Two-factor authentication (2FA) — a feature where web and native are essentially equal

Unlike biometrics or push notifications, 2FA is one of the few security features where the website and a native app would offer essentially the same experience, with no meaningful platform gap:

- Web / PWA — fully implementable today. Standard approaches (a time-based code from an authenticator app, or an email/SMS-based code) work identically whether the site is opened in a plain browser tab or as an installed PWA — nothing about the browser limits this.
- Native app — the same options are available, and could additionally integrate with an OS-level authenticator if the device has one, though this isn't meaningfully better than the web version for standard authenticator-app-based 2FA specifically.

In short: 2FA is worth considering purely on its own security merits, independent of the native-vs-web decision — it doesn't tip the scales toward building a native app the way push notifications or true offline support might.